

Worksheet — 2.2 Problem Solving & Programming — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

1. **Key-term match:** 1 → C; 2 → E; 3 → B; 4 → A; 5 → H; 6 → D; 7 → F; 8 → G.

2. **Scope (worked):** - a) **10** — the global `score`, before any subroutine runs. - b) **5** — the local `score` inside `tryLocal()`. - c) **99** — `addPoints()` assigned to the **global** `score`, so the change persists. - d) The `score` inside `tryLocal()` is a **local** variable that **shadows** the global only within that subroutine; it is destroyed when the subroutine ends, so it never affects the global value the final `print` reads.

3. **By value vs by reference (worked):** - a) `a` prints **1** — `increase` received a **copy** (by value), so the original was unchanged. - b) `b` prints **101** — `boost` received the **address** (by reference), so adding 100 changed the original. - c) The consequence: with **by value** the original variable is **unchanged** because only a copy is altered; with **by reference** the original **can be (and is) changed** because the subroutine works on the same memory location.

4. **Recursion trace:**

Call (frame pushed)	Base case?	Expression returned	Value returned (on pop)
factorial(4)	No	4 * factorial(3)	4 * 6 = 24
factorial(3)	No	3 * factorial(2)	3 * 2 = 6
factorial(2)	No	2 * factorial(1)	2 * 1 = 2
factorial(1)	No	1 * factorial(0)	1 * 1 = 1
factorial(0)	Yes	return 1	1

Frames are **pushed** down to `factorial(0)`, then **popped** back up in LIFO order, each multiplying its `n` by the value returned from below.

- (b) Final value printed: **24**.
- (c) Without the base case, `factorial(0)` would call `factorial(-1)`, then `factorial(-2)`, ... with **no stopping condition**. Each call **pushes another stack frame**; the call stack fills up and a **stack overflow** error occurs.

5. **Class (model answer):**

```

class Vehicle
  private make
  private numWheels

  public procedure new(m, w)
    make = m
    numWheels = w
  endprocedure

  public procedure describe()
    print("Make: " + make + ", wheels: " + numWheels)
  endprocedure
endclass

class Car inherits Vehicle
  private doors

  public procedure new(m, w, d)
    super.new(m, w)
    doors = d
  endprocedure

  public procedure describe()
    super.describe()
    print("Doors: " + doors)
  endprocedure
endclass

myCar = new Car("Ford", 4, 5)
myCar.describe()

```

Output:

```

Make: Ford, wheels: 4
Doors: 5

```

- b) **Encapsulation** (data hiding — private attributes accessed only via methods).
- c) **Polymorphism** via **overriding** (the subclass replaces the inherited method with the same name and parameters).

6. Computational methods: a) **Pipelining**; b) **Data mining**; c) **Heuristics**; d) **Backtracking**; e) **Performance modelling**; f) **Visualisation**.

7. Challenge (model answer):

```
procedure countdownR(n)
  if n == 0 then           // BASE CASE
    print("Liftoff")
  else                     // RECURSIVE CASE
    print(n)
    countdownR(n - 1)
  endif
endprocedure
```

- (ii) **Advantage** (any one): shorter/more elegant code that mirrors the self-similar structure; no explicit loop variable to manage. **Disadvantage** (any one): uses more memory (a stack frame per call); slower due to call overhead; risk of **stack overflow** for large `n`.